

Student Meal Planner

Learning Evidence: Self-Directed Learning

A Personal Leadership Document for the Grubs4Scrubs Project

Date: April 2026

Author: Yao Cheng Zhou

What This Document Covers

This is a record of the learning I did outside of class to build Grubs4Scrubs. The project required technologies I didn't know at the start of the semester: React, C#, ASP.NET Core, SQL Server, and ADO.NET. I had to teach myself most of this through online courses, official documentation, and trial and error in the actual project.

[NOTE: Add screenshots from your learning platforms (Codecademy, Microsoft Learn, YouTube courses, etc.) as proof. Each section below should ideally have a screenshot attached when you submit to Canvas.]

C# and .NET

What I used: Codecademy C# course, Microsoft Learn ASP.NET Core paths, official Microsoft documentation.

I came into the semester with no C# experience. I worked through Codecademy's C# course covering Hello World, Variables, Methods, Classes, Inheritance, Arrays, and started the Loops section. The syntax felt similar to Java, which helped, but the .NET ecosystem (NuGet packages, project files, Program.cs configuration) was entirely new.

What stuck with me most was understanding dependency injection. The concept made no sense from reading alone. It clicked when I registered `IRecipeRepository` with `RecipeRepository` in Program.cs and watched ASP.NET inject it into the service constructor automatically. That hands-on moment taught me more than any tutorial chapter.

Later in the semester, I had to learn security-related C# concepts for the authentication system: BCrypt password hashing with the BCrypt.Net-Next NuGet package, JWT token generation using `System.IdentityModel.Tokens.Jwt`, and claims-based identity with `System.Security.Claims`. None of this was covered in class. I learned it by reading the Microsoft Learn docs on JWT Bearer authentication and then building the AuthController step by step, testing each piece as I went.

[NOTE: Add your Codecademy/Microsoft Learn completion screenshots here.]

React and JavaScript

What I used: [Fill in: Codecademy React course, React official docs, YouTube, etc.]

React was the biggest learning curve. Components, props, state, hooks, routing. All new. [Describe your learning path.] I started with basic components and worked up to multi-page apps with React Router.

The concepts I struggled with most were `useState` (when does a re-render happen?), `useEffect` (still don't fully get the dependency array), and props (why do I pass data down instead of just importing it?). The props question got answered when I tried to share state between sibling components and realized why data flows down from a parent.

Some JavaScript concepts I had to learn along the way: `Object.values()` for iterating over object properties, array destructuring for cleaner variable assignment, and the difference between `.map()`, `.filter()`, and `.reduce()` (I ended up replacing reduce with a for loop because I couldn't follow what it was doing).

[NOTE: Add completion screenshots or certificates here.]

SQL Server and Database Management

What I used: [Fill in: Microsoft Learn SQL modules, W3Schools SQL, your database course materials, etc.]

I'd done basic SQL in a previous course, but connecting it to a real application was new. Setting up SQL Server, creating databases and tables through queries, and writing parameterized SQL for an API were all things I learned during this project.

The biggest lesson was about connection strings. I spent hours debugging "Error Locating Server/Instance Specified" because I had the wrong server format. Switched from LocalDB to full SQL Server, updated the connection string, and it worked. It's the kind of thing that's trivial once you know it but impossible to figure out from the error message alone.

[NOTE: Add any Microsoft Learn module completion screenshots here.]

How I Learn Best

Looking back at the semester, I learn best by building something and hitting errors. Reading documentation helps for understanding concepts, but the actual learning happens when I try to implement something and it breaks. The layered architecture didn't make sense until I moved files between projects and dealt with namespace errors. ADO.NET didn't click until I wrote a `SqlCommand` and mapped the results by hand.

For next semester, I want to be more intentional about combining structured learning (courses) with project work (building things). The courses give me vocabulary and concepts. The project gives me understanding.

Summary of Resources Used

[NOTE: Fill in with your actual resources and add links/screenshots]

Resource	Topic	Status
Codecademy C# Course	C# Basics (classes, methods, arrays, inheritance)	In progress (through Loops)
Microsoft Learn	ASP.NET Core, DI, ADO.NET, JWT auth, repository pattern	Referenced throughout
React Official Docs	Component composition, hooks, routing	Referenced throughout
MDN Web Docs	CSS variables, JavaScript methods, fetch API	Referenced throughout
YouTube tutorials	Specific concepts when docs weren't clicking	As needed
Stack Overflow	Bug-specific questions (connection strings, CORS, etc.)	As needed

[NOTE: Add any other platforms or resources you actually used. Don't list things you didn't touch.]